

Carrera: Ingeniería en sistemas de Información y ciencias de la computación.

Curso: Algoritmos

Docente: Moises Eduardo Velasquez Oliva



PRACTICA #1

Alumno: Marcos David Guerra Paiz

No. De carnet: 0910-25-15434

Ciclo: 2

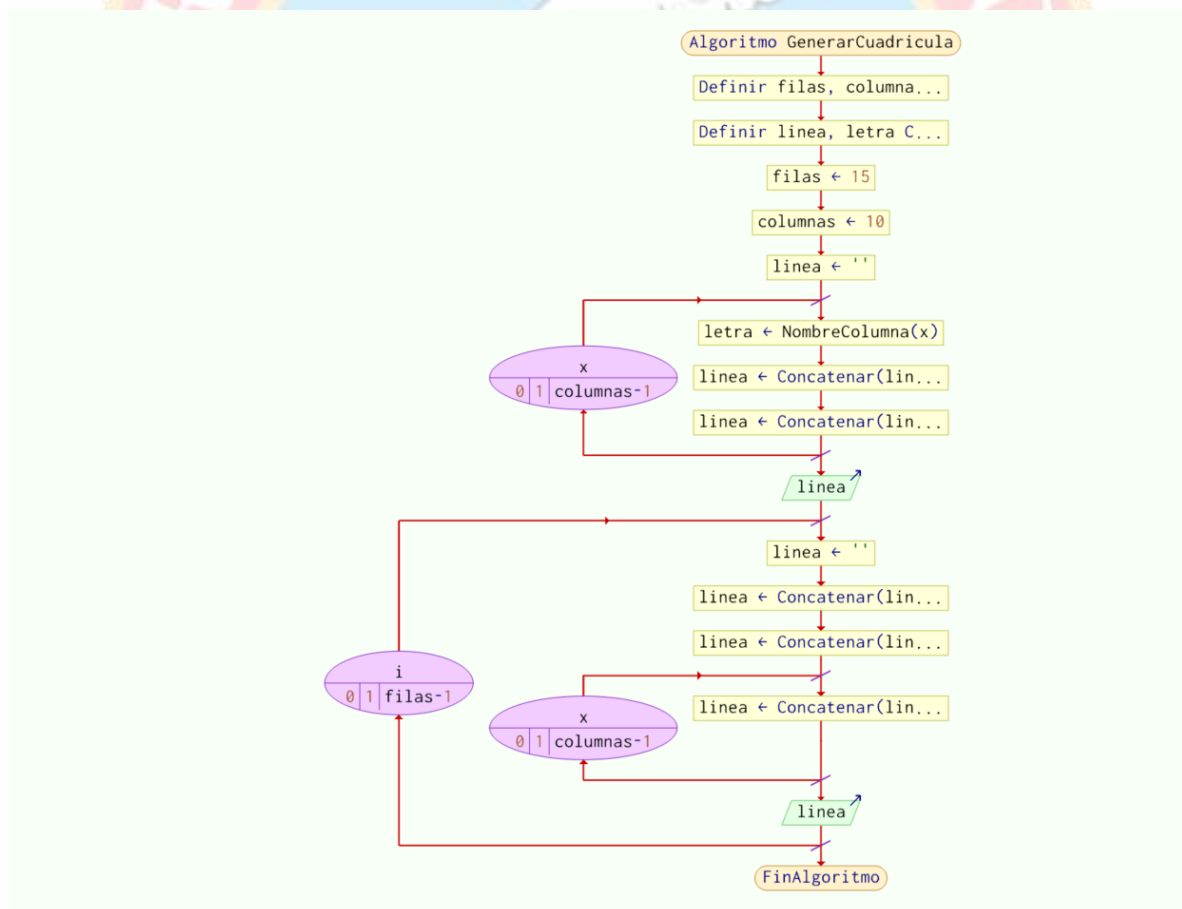
Sección: B

Explicación de la problemática en palabras propias

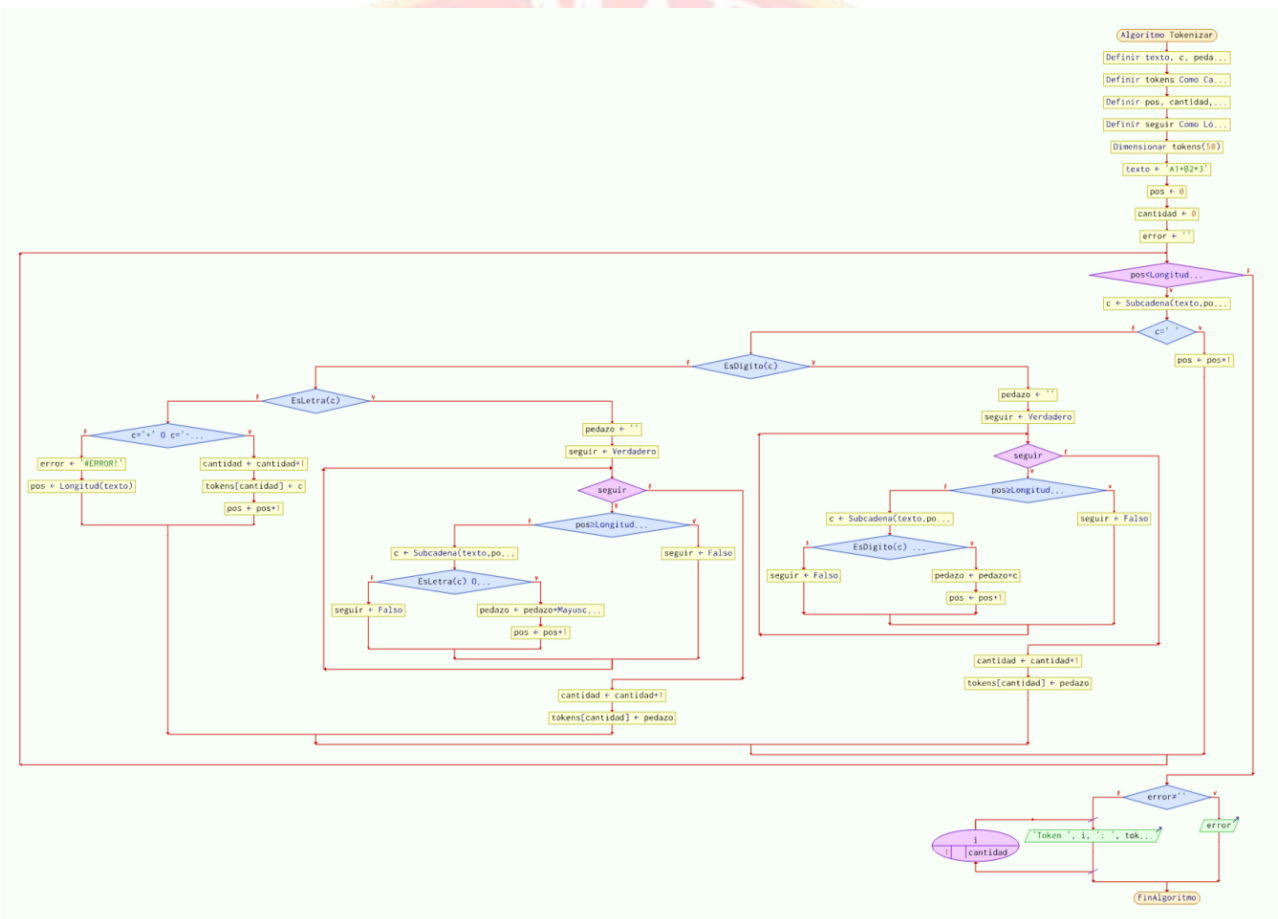
La Licda. Silvia pierde tiempo porque cada cambio en un dato la obliga a recalcular muchas celdas manualmente, lo que provoca errores y hasta pérdidas de dinero. Las herramientas en la nube no le sirven porque necesita trabajar sin internet, y las opciones comerciales no le permiten personalizar formatos como sus clientes requieren. En resumen, necesita una hoja de cálculo sencilla, local, adaptable y que recalcule automáticamente cuando cambia un valor.

5.2) Diagramas de flujo

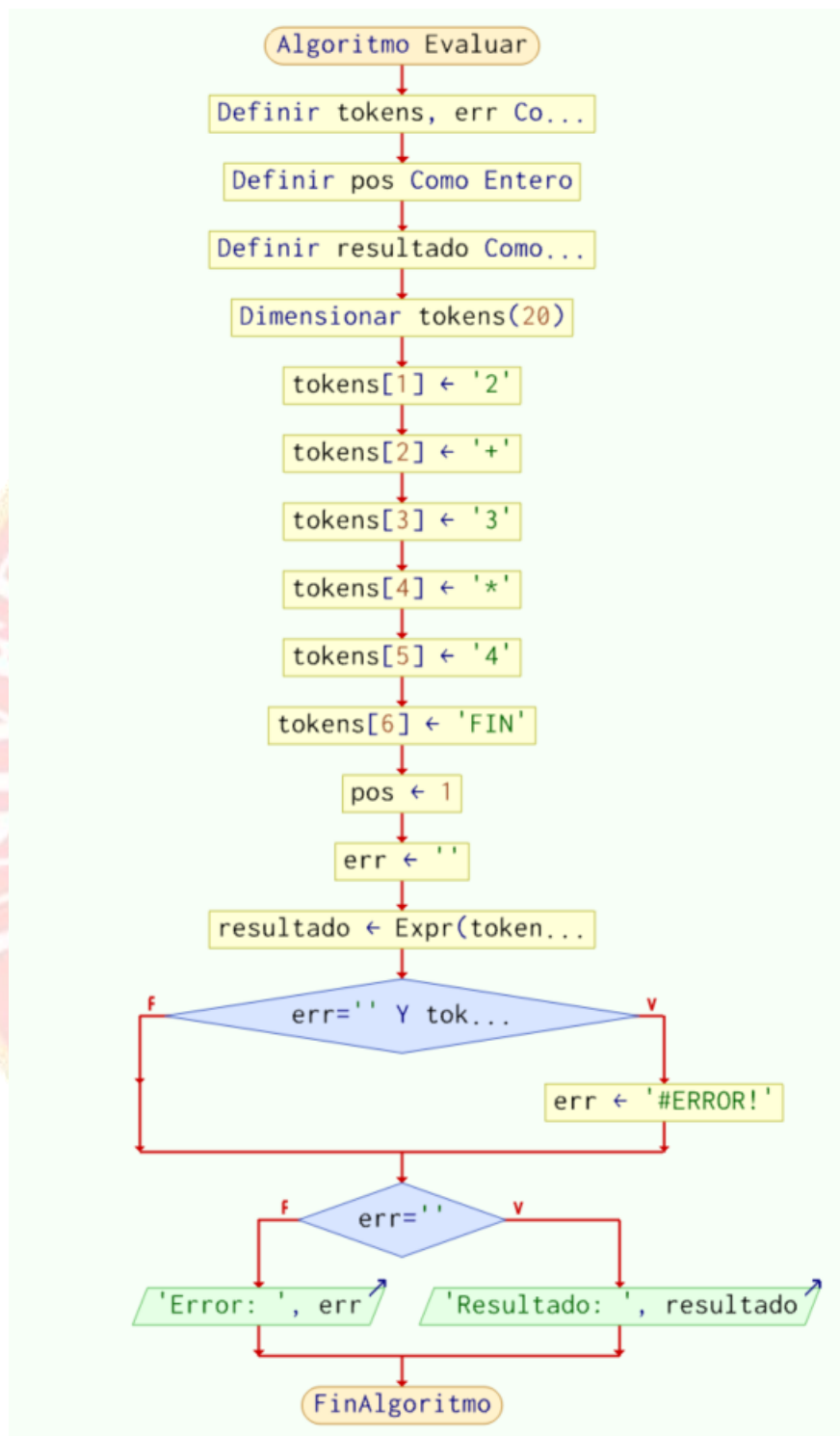
1. Generación de la cuadrícula (ciclos anidados para filas y columnas).



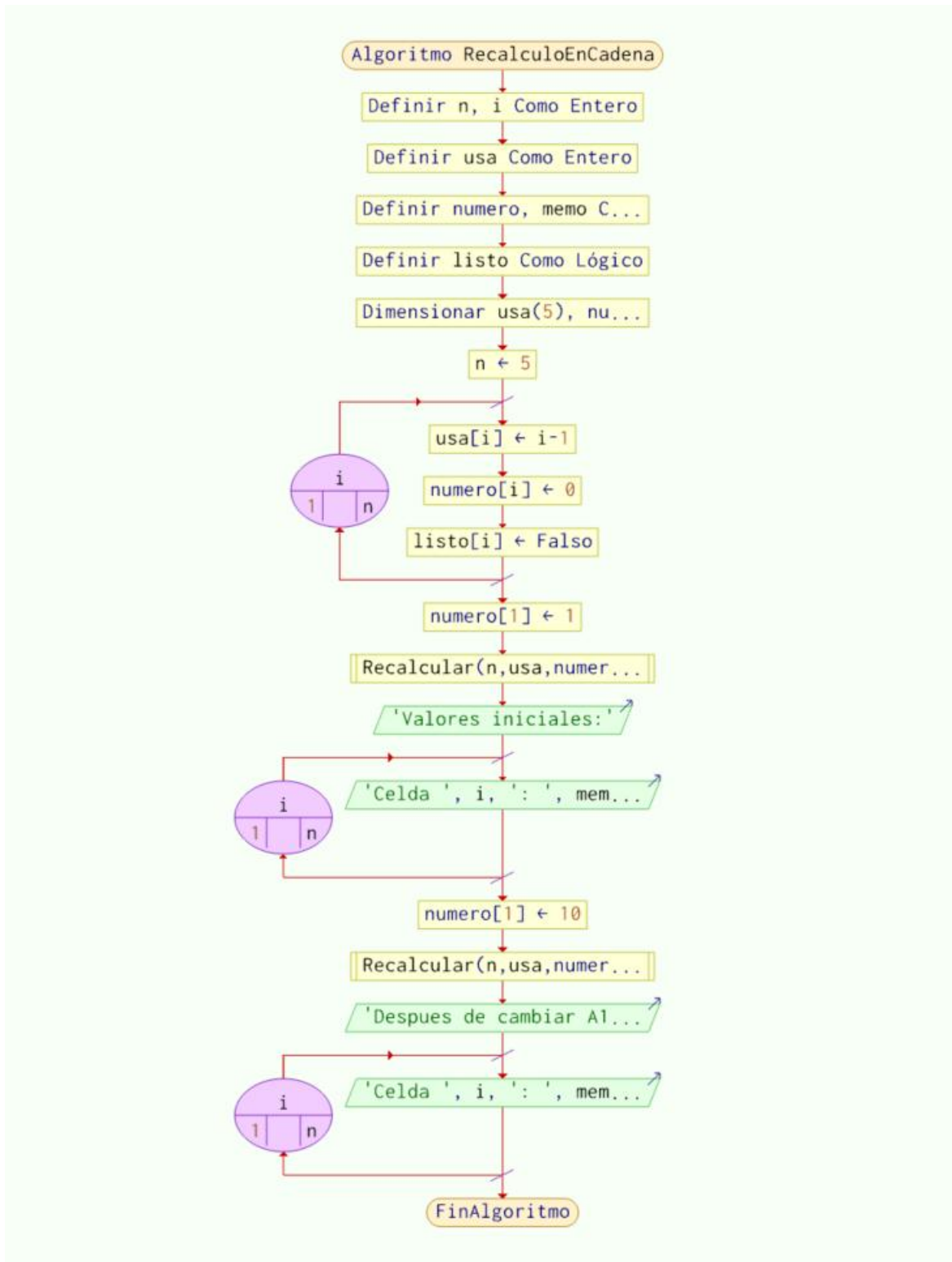
2. Tokenización de una fórmula (separar una cadena de texto como “=A1+B2*3” en sus componentes: números, referencias de celda, operadores y paréntesis).



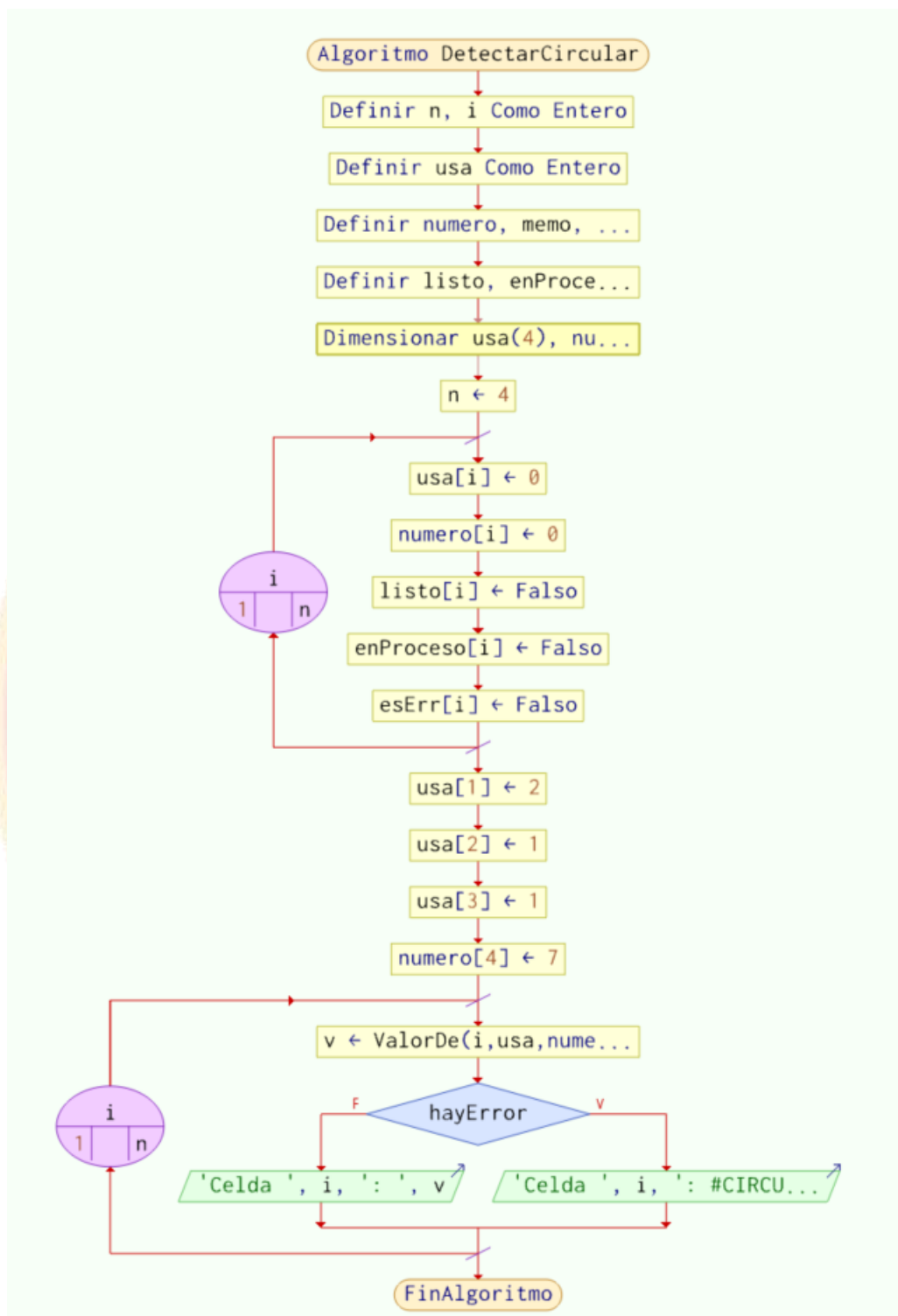
3. Evaluación de la expresión tokenizada, respetando precedencia de operadores.



4. Recálculo en cadena: dado que una celda cambió de valor, identificar y recalcular todas las celdas que dependen de ella.



5. Detección de referencias circulares.



5.3) Pseudocódigo

Algoritmo 1: Generación de la cuadrícula

Qué hace: genera la cuadrícula de la hoja. Primero escribe la fila de encabezados (A, B, C...) y luego, con dos ciclos anidados, escribe cada fila con su número y una celda por columna. El tamaño depende de dos valores: `filas` y `columnas`.

Correspondencia con el código JavaScript

Pseudocódigo	Código JavaScript (<code>construirTabla</code>)
<code>NombreColumna (x)</code>	<code>nombreColumna (x)</code> , que usa <code>String.fromCharCode (65 + x)</code>
Primer Para (encabezados)	Un for que crea un <code><th></code> por cada columna
Para <code>i</code> y Para <code>x</code> anidados	Dos for anidados: crean un <code><tr></code> por fila y un <code><td></code> por celda
Escribir línea	En JavaScript se agregan elementos a la tabla (<code>appendChild</code>) en lugar de escribir texto

```

1 SubProceso letra ← NombreColumna(x)
2   Definir letras Como Texto
3   letras ← "ABCDEFGHJKLMNOPQRSTUVWXYZ"
4   letra ← Subcadena(letras, x+1, x+1)
5 FinSubProceso
6
7 Proceso GenerarCuadrícula
8   Definir filas, columnas, x, i Como Entero
9   Definir línea, letra Como Texto
10
11   filas ← 15
12   columnas ← 10
13
14   línea ← ""
15   Para x ← 0 Hasta columnas-1 Con Paso 1 Hacer
16   |   letra ← NombreColumna(x)
17   |   línea ← Concatenar(línea, letra)
18   |   línea ← Concatenar(línea, " ")
19   FinPara
20   Escribir línea
21
22   Para i ← 0 Hasta filas-1 Con Paso 1 Hacer
23   |   línea ← ""
24   |   línea ← Concatenar(línea, ConvertirATexto(i+1))
25   |   línea ← Concatenar(línea, " ")
26   |   Para x ← 0 Hasta columnas-1 Con Paso 1 Hacer
27   |   |   línea ← Concatenar(línea, "[ ] ")
28   |   FinPara
29   |   Escribir línea
30   FinPara
31 FinProceso

```

Explicación

1. **NombreColumna(x)** convierte un número en una letra: guarda el alfabeto en un texto y toma la letra que está en la posición x (0 es A, 1 es B, 2 es C...).
2. **Tamaño:** $\text{filas} \leftarrow 15$ y $\text{columnas} \leftarrow 10$. Para cambiar el tamaño de la hoja solo se cambian estos dos números.
3. **Encabezados:** el primer **Para** recorre las columnas de 0 a $\text{columnas} - 1$ y va juntando las letras en la variable **línea**. Al terminar se escribe la línea.
4. **Ciclos anidados:** el **Para** de afuera (y) recorre las filas. En cada vuelta empieza la línea con el número de fila ($y + 1$, porque las filas se numeran desde 1). El **Para** de adentro (x) recorre las columnas y agrega una celda $[]$. Por cada vuelta de y , el ciclo de x da una vuelta completa.
5. **Resultado:** con 15 filas y 10 columnas se crean 150 celdas escribiendo las instrucciones una sola vez.

Algoritmo 2: Tokenización de una fórmula

Qué hace: separa el texto de una fórmula (sin el signo =) en piezas llamadas tokens: números, nombres y símbolos. Por ejemplo, $A1+B2*3$ se convierte en $A1$, $+$, $B2$, $*$, 3 .

Correspondencia con el código JavaScript

Pseudocódigo	Código JavaScript (tokenizar)
EsDigito(c), EsLetra(c)	esDigito(c), esLetra(c)
Mientras pos < Longitud(texto)	while (pos < texto.length)
Ciclos internos con seguir	Ciclos while que arman el número o el nombre
tokens[cantidad] ← pedazo	tokens.push(...)
error ← "#ERROR!"	throw new Error('#ERROR!')

```

1  SubProceso r ← EsDigito(c)
2      Definir r Como Logico
3      r ← (c ≥ "0" Y c ≤ "9")
4  FinSubProceso
5
6  SubProceso r ← EsLetra(c)
7      Definir r Como Logico
8      r ← (Mayusculas(c) ≥ "A" Y Mayusculas(c) ≤ "Z")
9  FinSubProceso
10
11 Proceso Tokenizar
12     Definir texto, c, pedazo, error Como Caracter
13     Definir tokens Como Caracter
14     Definir pos, cantidad, i Como Entero
15     Definir seguir Como Logico
16     Dimension tokens[50]
17
18     texto ← "A1+B2*3"
19     pos ← 0
20     cantidad ← 0
21     error ← ""
22
23     Mientras pos < Longitud(texto) Hacer
24         c ← Subcadena(texto, pos, pos)
25
26         Si c = " " Entonces
27             pos ← pos + 1
28         Sino
29             Si EsDigito(c) Entonces
30                 pedazo ← ""
31                 seguir ← Verdadero
32                 Mientras seguir Hacer
33                     Si pos ≥ Longitud(texto) Entonces
34                         seguir ← Falso
35                     Sino
36                         c ← Subcadena(texto, pos, pos)
37                         Si EsDigito(c) O c = "." Entonces
38                             pedazo ← pedazo + c
39                             pos ← pos + 1
40                         Sino
41                             seguir ← Falso
42                         FinSi
43                     FinSi
44                 FinMientras
45                 cantidad ← cantidad + 1
46                 tokens[cantidad] ← pedazo
47             Sino

```

```

47     Sino
48     Si EsLetra(c) Entonces
49     |
50     |   pedazo ← ""
51     |   seguir ← Verdadero
52     |   Mientras seguir Hacer
53     |   |   Si pos ≥ Longitud(texto) Entonces
54     |   |   |   seguir ← Falso
55     |   |   Sino
56     |   |   |   c ← Subcadena(texto, pos, pos)
57     |   |   |   Si EsLetra(c) O EsDigito(c) Entonces
58     |   |   |   |   pedazo ← pedazo + Mayusculas(c)
59     |   |   |   |   pos ← pos + 1
60     |   |   |   Sino
61     |   |   |   |   seguir ← Falso
62     |   |   FinSi
63     |   FinSi
64     FinMientras
65     cantidad ← cantidad + 1
66     tokens[cantidad] ← pedazo
67     Sino
68     Si c = "+" O c = "-" O c = "*" O c = "/" O c = "(" O c = ")" O c = "," O c = ":" O c = ";" Entonces
69     |
70     |   cantidad ← cantidad + 1
71     |   tokens[cantidad] ← c
72     |   pos ← pos + 1
73     Sino
74     |
75     |   error ← "#ERROR!"
76     |   pos ← Longitud(texto)
77     FinSi
78   FinSi
79 FinSi
80 FinSi
81 FinMientras
82
83 Si error ≠ "" Entonces
84   Escribir error
85 Sino
86   Para i ← 1 Hasta cantidad Hacer
87   |   Escribir "Token ", i, ": ", tokens[i]
88   FinPara
89 FinSi
90 FinProceso

```

Explicación

1. **Variables:** `texto` es la fórmula, `pos` es la posición del carácter que se está leyendo (empieza en 0), `cantidad` cuenta los tokens guardados y `error` guarda un mensaje si algo sale mal.
2. **Ciclo principal:** mientras no se llegue al final del texto, toma el carácter `c` que está en `pos`.
3. **Espacio:** se ignora (`pos` avanza una posición).
4. **Dígito:** arma un número completo mientras siga habiendo dígitos o puntos, así `12.5` queda como un solo token. Usa un ciclo interno con la variable `seguir`.

5. **Letra:** arma un nombre completo mientras siga habiendo letras o dígitos y lo pasa a mayúsculas (A1, SUMA). Todavía no se decide si es una celda o una función: eso lo hace el evaluador.
6. **Símbolo:** los caracteres + - * / () , : ; se guardan tal cual.
7. **Cualquier otro carácter** (por ejemplo \$) no está permitido: se guarda el error #ERROR! y el ciclo termina.

Al final: si hubo error se muestra; si no, se escriben todos los tokens.

Algoritmo 3: Evaluación respetando la precedencia

Qué hace: calcula el valor de una expresión ya tokenizada respetando el orden matemático: primero los paréntesis, luego las multiplicaciones y divisiones, y al final las sumas y restas. También maneja el signo negativo (-5).

Función	Qué reconoce	Operadores
Expr	Sumas y restas	+ -
Term	Multiplicaciones y divisiones	* /
Unario	Un signo delante de un valor	+ -
Primario	Un número o una expresión entre paréntesis	()

Correspondencia con el código JavaScript

Pseudocódigo	Código JavaScript (evaluar)
Expr, Term, Unario, Primario	Las funciones <code>expr()</code> , <code>term()</code> , <code>unario()</code> y <code>primario()</code> dentro de <code>evaluar()</code>
Variable <code>err</code>	En JavaScript el error se lanza con <code>throw new Error(...)</code> y se atrapa con <code>try/catch</code> en <code>calcularTexto()</code>
Token "FIN"	En JavaScript, al terminar la lista <code>tokens[pos]</code> es <code>undefined</code>
(no incluido)	El código real también lee referencias a celdas (A1), rangos (A1:A3) y funciones (SUMA) dentro de <code>primario()</code>

```

1 SubProceso ok ← EsNumero(t)
2   Definir ok Como Logico
3   ok ← (Subcadena(t, 0, 0) ≥ "0" Y Subcadena(t, 0, 0) ≤ "9")
4 FinSubProceso
5
6 SubProceso r ← Primario(tokens, pos Por Referencia, err Por Referencia)
7   Definir r Como Real
8   Definir t Como Caracter
9   t ← tokens[pos]
10  r ← 0
11  Si t = "(" Entonces
12    pos ← pos + 1
13    r ← Expr(tokens, pos, err)
14    Si tokens[pos] = ")" Entonces
15      pos ← pos + 1
16    Sino
17      err ← "#ERROR!"
18    FinSi
19  Sino
20    Si EsNumero(t) Entonces
21      r ← ConvertirANumero(t)
22      pos ← pos + 1
23    Sino
24      err ← "#ERROR!"
25    FinSi
26  FinSi
27 FinSubProceso
28
29 SubProceso r ← Unario(tokens, pos Por Referencia, err Por Referencia)
30   Definir r Como Real
31   Si tokens[pos] = "-" Entonces
32     pos ← pos + 1
33     r ← 0 - Unario(tokens, pos, err)
34   Sino
35     Si tokens[pos] = "+" Entonces
36       pos ← pos + 1
37       r ← Unario(tokens, pos, err)
38     Sino
39       r ← Primario(tokens, pos, err)
40     FinSi
41   FinSi
42 FinSubProceso

```

```

43
44 SubProceso r ← Term(tokens, pos Por Referencia, err Por Referencia)
45   Definir r, derecho Como Real
46   Definir op Como Caracter
47   r ← Unario(tokens, pos, err)
48   Mientras tokens[pos] = "*" O tokens[pos] = "/" Hacer
49     op ← tokens[pos]
50     pos ← pos + 1
51     derecho ← Unario(tokens, pos, err)
52     Si op = "*" Entonces
53       r ← r * derecho
54     Sino
55       Si derecho = 0 Entonces
56         err ← "#DIV/0!"
57       Sino
58         r ← r / derecho
59     FinSi
60   FinSi
61   FinMientras
62 FinSubProceso
63
64 SubProceso r ← Expr(tokens, pos Por Referencia, err Por Referencia)
65   Definir r, derecho Como Real
66   Definir op Como Caracter
67   r ← Term(tokens, pos, err)
68   Mientras tokens[pos] = "+" O tokens[pos] = "-" Hacer
69     op ← tokens[pos]
70     pos ← pos + 1
71     derecho ← Term(tokens, pos, err)
72     Si op = "+" Entonces
73       r ← r + derecho
74     Sino
75       r ← r - derecho
76     FinSi
77   FinMientras
78 FinSubProceso
79
80 Proceso Evaluar
81   Definir tokens, err Como Caracter
82   Definir pos Como Entero
83   Definir resultado Como Real
84   Dimension tokens[20]
85
86
87   tokens[1] ← "2"
88   tokens[2] ← "+"
89   tokens[3] ← "3"
90   tokens[4] ← "*"
91   tokens[5] ← "4"
92   tokens[6] ← "FIN"
93
94   pos ← 1
95   err ← ""
96   resultado ← Expr(tokens, pos, err)
97
98
99   Si err = "" Y tokens[pos] ≠ "FIN" Entonces
100     err ← "#ERROR!"
101   FinSi
102
103   Si err = "" Entonces
104     Escribir "Resultado: ", resultado
105   Sino
106     Escribir "Error: ", err
107   FinSi
108 FinProceso
109

```

Explicación

1. **Programa principal:** recibe los tokens (con "FIN" al final), pone `pos ← 1` y llama a `Expr`. Después revisa que no hayan sobrado tokens: si `tokens[pos]` no es "FIN", la fórmula estaba mal escrita y el resultado es #ERROR! (por ejemplo `1+2)`).
2. **`Expr`:** pide un `Term`. Mientras vea un `+` o un `-`, lee otro `Term` y suma o resta. Lo hace de izquierda a derecha, así que `10-2-3` da 5.
3. **`Term`:** igual que `Expr`, pero con `Unario` y los operadores `*` y `/`. Antes de dividir comprueba que el divisor no sea 0; si lo es, el error es #DIV/0!.
4. **`Unario`:** si ve un `-`, avanza y devuelve el valor con el signo cambiado (`0 - ...`). Si ve un `+`, lo salta. Si no hay signo, llama a `Primario`.
5. **`Primario`:** si el token es `(`, calcula una `Expr` completa y exige que después venga un `)`. Si es un número, lo convierte con `ConvertirANumero`. Cualquier otra cosa es #ERROR!.
6. **Por qué respeta la precedencia:** `Expr` llama a `Term`, y `Term` resuelve las multiplicaciones y divisiones antes de devolver el resultado. Así, la multiplicación siempre ocurre antes que la suma. Los paréntesis funcionan porque `Primario` vuelve a llamar a `Expr`.

Algoritmo 4: Recálculo en cadena

Qué hace: cuando una celda cambia, la hoja se recalcula para que todas las celdas que dependen de ella queden actualizadas. Cada celda pide primero el valor de la celda que usa y guarda su resultado en `memo` para no calcularlo dos veces.

Modelo del ejemplo: para que el algoritmo sea corto, cada celda usa como máximo otra celda y su fórmula es "(celda usada) + 1". En el ejemplo, A1 vale 1, B1 es `=A1+1`, C1 es `=B1+1`, D1 es `=C1+1` y E1 es `=D1+1`.

Correspondencia con el código JavaScript

Pseudocódigo	Código JavaScript
Recalcular	<code>recalcular()</code> , que se llama desde <code>mostrar()</code> cada vez que cambia un dato
<code>ValorDe(i, ...)</code>	<code>valorDe(id)</code> dentro de <code>recalcular()</code>
<code>memo[i], listo[i]</code>	<code>memo</code> , un Map con los resultados ya calculados
<code>usa[i]</code>	Las referencias (A1, B2...) que aparecen dentro de la fórmula

Pseudocódigo	Código JavaScript
numero[i]	El valor fijo escrito en la celda

```

1  SubProceso v ← ValorDe(i, usa, numero, memo, listo)
2      Definir v Como Real
3      Si listo[i] Entonces
4          v ← memo[i]
5      Sino
6          Si usa[i] = 0 Entonces
7              v ← numero[i]
8          Sino
9              v ← ValorDe(usa[i], usa, numero, memo, listo) + 1
10         FinSi
11         memo[i] ← v
12         listo[i] ← Verdadero
13     FinSi
14 FinSubProceso
15
16 SubProceso Recalcular(n, usa, numero, memo, listo)
17     Definir i Como Entero
18     Definir v Como Real
19     Para i ← 1 Hasta n Hacer
20         listo[i] ← Falso
21     FinPara
22     Para i ← 1 Hasta n Hacer
23         v ← ValorDe(i, usa, numero, memo, listo)
24     FinPara
25 FinSubProceso
26
27 Proceso RecalculoEnCadena
28     Definir n, i Como Entero
29     Definir usa Como Entero
30     Definir numero, memo Como Real
31     Definir listo Como Logico
32     Dimension usa[5], numero[5], memo[5], listo[5]
33
34     n ← 5
35     Para i ← 1 Hasta n Hacer
36         usa[i] ← i - 1
37         numero[i] ← 0
38         listo[i] ← Falso
39     FinPara
40     numero[1] ← 1
41
42     Recalcular(n, usa, numero, memo, listo)
43     Escribir "Valores iniciales:"
44     Para i ← 1 Hasta n Hacer
45         Escribir "Celda ", i, ": ", memo[i]
46     FinPara
47
48
49     numero[1] ← 10
50     Recalcular(n, usa, numero, memo, listo)
51     Escribir "Despues de cambiar A1 a 10:"
52     Para i ← 1 Hasta n Hacer
53         Escribir "Celda ", i, ": ", memo[i]
54     FinPara
55 FinProceso
56

```

Explicación

1. **`Recalcular`**: primero marca todas las celdas como "no calculadas" (`listo ← Falso`) y luego pide el valor de cada una con `ValorDe`.
2. **`ValorDe(i)`**: si la celda ya está calculada, devuelve el valor guardado en `memo`. Si no, mira si usa otra celda. Si no usa ninguna, su valor es el número fijo. Si usa otra, **primero pide el valor de esa celda** (llamada recursiva), le suma 1, guarda el resultado en `memo` y marca la celda como calculada.
3. **Por qué quedan actualizadas las dependientes**: como cada celda pide primero las celdas que usa, no importa en qué orden se recorran. Una celda dependiente siempre usa valores ya actualizados.
4. **Decisión de diseño**: el programa no busca una por una qué celdas dependen de la que cambió, sino que recalcula toda la hoja. El resultado es el mismo y es más simple y menos propenso a errores de orden. Con 150 celdas el cálculo es instantáneo.

Algoritmo 5: Detección de referencias circulares

Qué hace: detecta las referencias circulares (por ejemplo, A1 usa B1 y B1 usa A1) para que el cálculo no se quede dando vueltas. Es el algoritmo 4 con una protección extra: la lista `enProceso`.

Correspondencia con el código JavaScript

Pseudocódigo	Código JavaScript
<code>enProceso[i]</code>	<code>enProceso</code> , un Set dentro de <code>recalcular()</code>
<code>hayError ← Verdadero (ciclo)</code>	<code>return { error: '#CIRCULAR!' }</code>
<code>esErr[i]</code>	Una celda cuyo valor es un objeto <code>{ error: ... }</code>
Escribir <code>"#CIRCULAR!"</code>	<code>mostrar()</code> pinta la celda en rojo y escribe el aviso "Referencia circular en: ..."

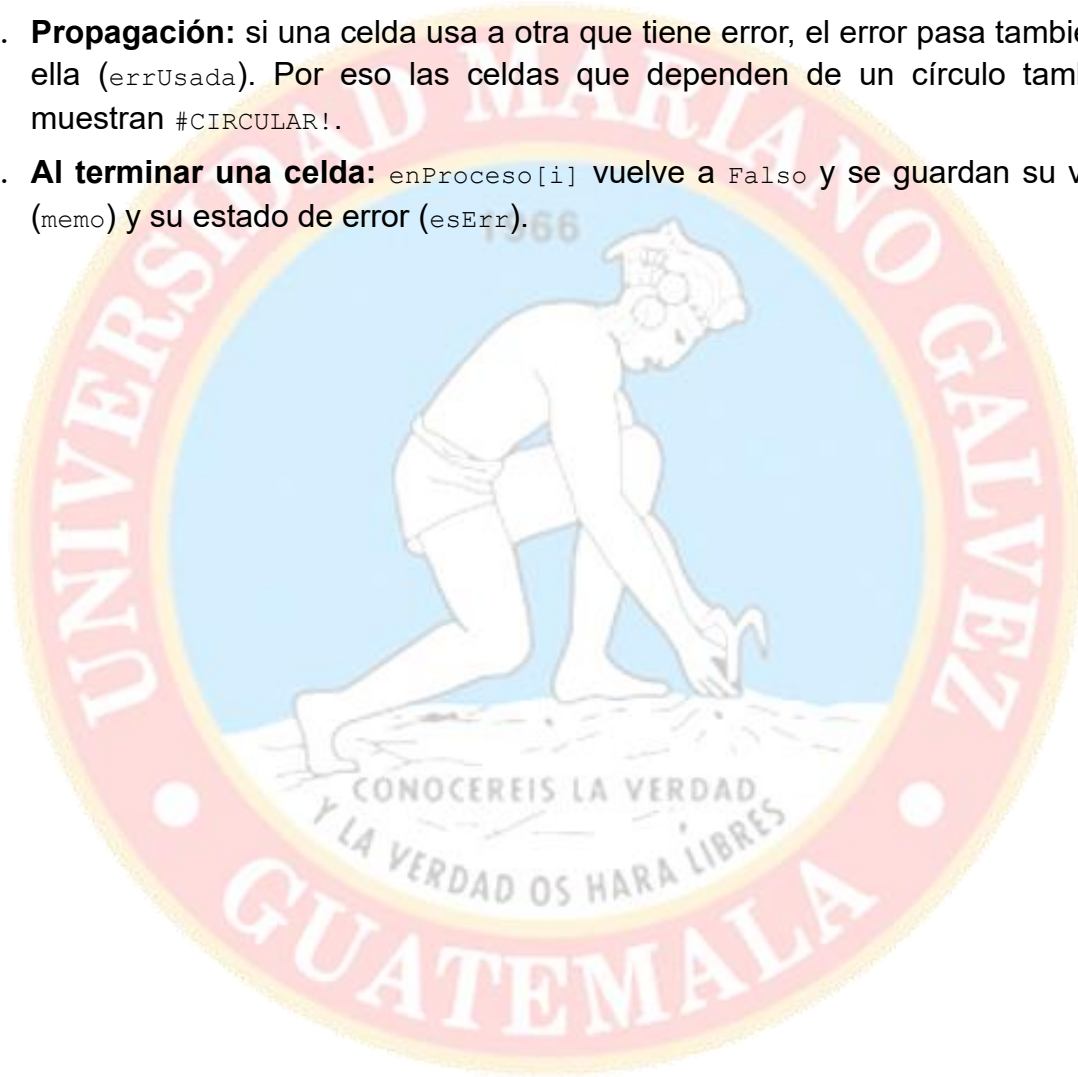
```

1  SubProceso v ← ValorDe(i, usa, numero, memo, listo, enProceso, esErr, hayError Por Referencia)
2      Definir v, vUsada Como Real
3      Definir errUsada Como Logico
4      hayError ← Falso
5      v ← 0
6      Si listo[i] Entonces
7          v ← memo[i]
8          hayError ← esErr[i]
9      Sino
10         Si enProceso[i] Entonces
11             hayError ← Verdadero
12         Sino
13             enProceso[i] ← Verdadero
14             Si usa[i] = 0 Entonces
15                 v ← numero[i]
16             Sino
17                 vUsada ← ValorDe(usa[i], usa, numero, memo, listo, enProceso, esErr, errUsada)
18                 Si errUsada Entonces
19                     hayError ← Verdadero
20                 Sino
21                     v ← vUsada + 1
22                 FinSi
23             FinSi
24             enProceso[i] ← Falso
25             memo[i] ← v
26             esErr[i] ← hayError
27             listo[i] ← Verdadero
28         FinSi
29     FinSi
30 FinSubProceso
31
32 Proceso DetectarCircular
33     Definir n, i Como Entero
34     Definir usa Como Entero
35     Definir numero, memo, v Como Real
36     Definir listo, enProceso, esErr, hayError Como Logico
37     Dimension usa[4], numero[4], memo[4], listo[4], enProceso[4], esErr[4]
38
39     n ← 4
40     Para i ← 1 Hasta n Hacer
41         usa[i] ← 0
42         numero[i] ← 0
43         listo[i] ← Falso
44         enProceso[i] ← Falso
45         esErr[i] ← Falso
46     FinPara
47
48     usa[1] ← 2
49     usa[2] ← 1
50     usa[3] ← 1
51     numero[4] ← 7
52
53     Para i ← 1 Hasta n Hacer
54         v ← ValorDe(i, usa, numero, memo, listo, enProceso, esErr, hayError)
55         Si hayError Entonces
56             Escribir "Celda ", i, ": #CIRCULAR!"
57         Sino
58             Escribir "Celda ", i, ": ", v
59         FinSi
60     FinPara
61 FinProceso
62

```

Explicación

1. **La idea:** mientras se calcula una celda, se marca en `enProceso`. Si durante ese cálculo se vuelve a pedir la misma celda, significa que su fórmula depende de sí misma: hay un círculo.
2. **Qué se hace al detectarlo:** se activa `hayError`, no se sigue calculando y se devuelve el control. Como el ciclo se corta en el momento en que la celda se pide por segunda vez, el programa nunca se queda en un ciclo infinito.
3. **Propagación:** si una celda usa a otra que tiene error, el error pasa también a ella (`errUsada`). Por eso las celdas que dependen de un círculo también muestran `#CIRCULAR!`.
4. **Al terminar una celda:** `enProceso[i]` vuelve a `Falso` y se guardan su valor (`memo`) y su estado de error (`esErr`).



5.4 Diseño de la estructura de datos

La hoja se representa con **un objeto con claves tipo "A1"**: cada clave es el nombre de una celda y su valor es el texto que escribió el usuario. Esta estructura se llama `datos`. Es independiente de la tabla que se ve en pantalla: la tabla solo muestra lo que sale de calcular `datos`.

Estructuras principales

Variable	Tipo	Qué contiene	Ejemplo
<code>datos</code>	Objeto (nombre → texto)	Lo que escribió el usuario, tal cual (valores y fórmulas). Las celdas vacías no existen en el objeto.	<code>datos["B1"] = "=A3*2"</code>
<code>celdas</code>	Arreglo de arreglos	Los elementos <code><td></code> de la tabla, para encontrar una celda sin buscarla en el DOM.	<code>celdas[0][0]</code> es la celda A1
<code>calculado</code>	Map (nombre → resultado)	El resultado de cada celda después de recalcar.	<code>calculado.get("B1")</code> da 60
<code>memo</code>	Map (dentro de recalcar)	Resultados ya calculados en este recálculo, para no repetirlos.	
<code>enProceso</code>	Set (dentro de recalcar)	Celdas que se están calculando en este momento, para detectar ciclos.	
<code>activa</code>	Objeto {x, y}	La celda seleccionada con el teclado.	{x: 0, y: 0} es A1
<code>editando</code>	Objeto o null	La celda que se está editando (null si ninguna).	null

Por qué un objeto con claves

- **Coincide con cómo se nombran las celdas** en las fórmulas: `datos["A1"]` es directamente lo que el usuario escribe como A1.
- **No ocupa espacio por las celdas vacías**: solo existen las celdas que tienen algo.
- **Se guarda fácil**: un objeto con textos se convierte a JSON y se guarda en `localStorage` sin cambios.

- **Costo:** hace falta convertir coordenadas en nombres. Eso lo hacen `idCelda(x, y)` (la posición (0, 0) da "A1") y `leerId("A1")` (hace lo contrario).

Datos escritos y valores calculados

El programa guarda el texto escrito y calcula el resultado aparte. Así, al editar una celda se recupera la fórmula original, y no solo el valor.

Celda	datos (lo escrito)	calculado (lo que se muestra)
A1	"10"	10
A2	"20"	20
A3	"=SUMA(A1:A2) "	30
B1	"=A3*2"	60
C1	"hola"	"hola"
D1	(no existe en datos)	null: la celda se ve vacía
E1	"=1/0"	{ error: "#DIV/0!" }

Tipos de resultado

Tipo	Ejemplo	Cómo se muestra
Número	30	Alineado a la derecha; en rojo si es negativo
Texto	"hola"	Tal cual, alineado a la izquierda
Error	{ error: "#DIV/0!" }	El código del error, en rojo
Vacío	null	Celda en blanco

Dependencias entre celdas

El programa no guarda una lista aparte de "qué celda depende de cuál". Las dependencias **están dentro de las fórmulas**: al calcular una celda, el evaluador lee las referencias que contiene (A1, B2, A1:A3) y pide el valor de esas celdas. Así, las dependencias se descubren al calcular y forman un árbol de llamadas.

En el ejemplo de la Figura 13, para calcular B1 ($=A3*2$) hace falta A3; para calcular A3 ($=SUMA(A1:A2)$) hacen falta A1 y A2.

- **`memo`**: guarda los resultados ya calculados, para que una celda que usan varias fórmulas se calcule una sola vez.
- **`enProceso`**: guarda las celdas que se están calculando. Si se vuelve a pedir una de ellas, hay una referencia circular y esas celdas muestran **#CIRCULAR!**.

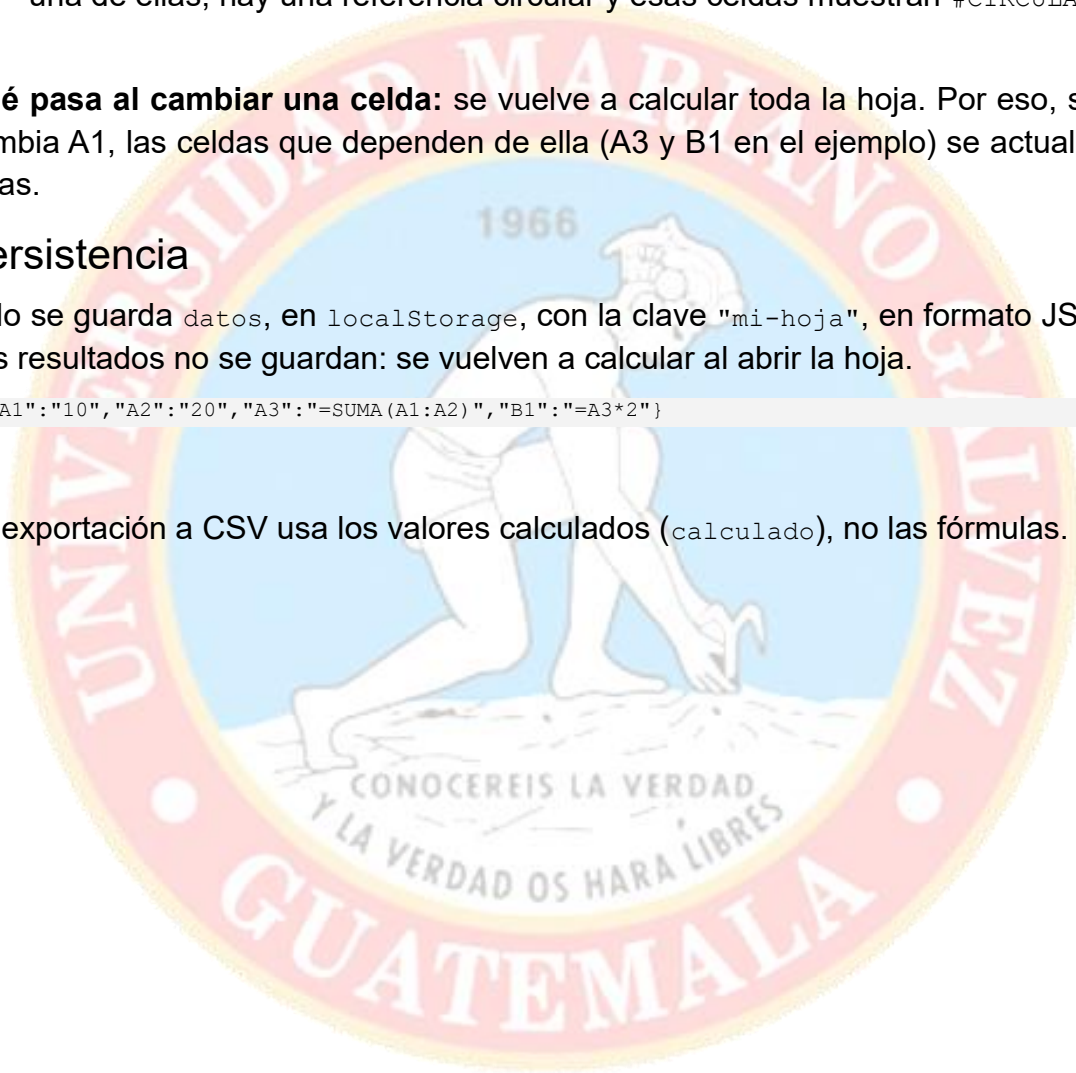
Qué pasa al cambiar una celda: se vuelve a calcular toda la hoja. Por eso, si se cambia A1, las celdas que dependen de ella (A3 y B1 en el ejemplo) se actualizan solas.

Persistencia

Solo se guarda **datos**, en `localStorage`, con la clave "mi-hoja", en formato JSON. Los resultados no se guardan: se vuelven a calcular al abrir la hoja.

```
{"A1": "10", "A2": "20", "A3": "=SUMA(A1:A2)", "B1": "=A3*2"}
```

La exportación a CSV usa los valores calculados (**calculado**), no las fórmulas.



5.5 Manual de usuario básico

Este manual explica, paso a paso y sin términos técnicos, cómo usar la hoja de cálculo.

1. ¿Para qué sirve?

Es una hoja de cálculo sencilla, parecida a Excel. Sirve para escribir números y textos en cuadritos (llamados celdas) y hacer cuentas automáticas con ellos.

2. Cómo abrirla

1. Busque el archivo **hoja-final-logo.html**.
2. Haga doble clic sobre él. Se abre en su navegador (Edge, Chrome o Firefox). No necesita internet.
3. Si quiere ver el logo de Excel en la esquina, el archivo `logo-excel.png` debe estar en la misma carpeta.

3. Conocer la pantalla

Parte	Qué es
Letras (A a J)	Los nombres de las columnas
Números (1 a 15)	Los nombres de las filas
Celda	Cada cuadrito. Su nombre es la letra de la columna y el número de la fila. Por ejemplo, B3 es la columna B, fila 3.
Recuadro azul	Indica la celda seleccionada
Botón Exportar CSV	Descarga los resultados en un archivo que se abre en Excel
Botón Limpiar hoja	Borra todo (antes pregunta si está seguro)
Aviso en rojo (arriba de la tabla)	Aparece cuando hay un problema de referencia circular

4. Escribir un valor

1. Haga clic en la celda que quiera.
2. Haga doble clic, o presione **Enter**, o simplemente empiece a escribir.
3. Escriba el número o el texto.

4. Presione **Enter** para guardar (baja a la celda de abajo). Si se arrepiente, presione **Esc** para cancelar.

Para **borrar** una celda, selecciónela y presione **Supr** (o Retroceso).

Importante: los decimales se escriben con punto. Por ejemplo, 3.5 y no 3,5.

5. Escribir una fórmula

Una fórmula es una cuenta. **Siempre empieza con el signo =**. La celda muestra el resultado, y si usted vuelve a editarla, verá la fórmula.

Escribe	Resultado
=2+3	5
=A1+B1	La suma de lo que hay en A1 y en B1
=A1*2	El doble de lo que hay en A1
=(A1+A2)/2	El promedio de A1 y A2
=2+3*4	14 (primero se multiplica y luego se suma)
=(2+3)*4	20 (lo que está entre paréntesis se hace primero)

Se pueden usar + (sumar), - (restar), * (multiplicar), / (dividir) y paréntesis. Si una fórmula usa otras celdas, **el resultado cambia solo** cuando usted cambia esas celdas.

6. Usar funciones

Las funciones hacen cuentas con varias celdas a la vez. Un **rango** se escribe con dos puntos: A1:A5 significa las celdas A1, A2, A3, A4 y A5.

Escribe	Qué hace
=SUMA (A1:A5)	Suma los números de A1 a A5
=PROMEDIO (A1:A5)	Calcula el promedio de esos números
=MAX (A1:A5)	Muestra el número más grande
=MIN (A1:A5)	Muestra el número más pequeño

Las celdas vacías o con texto se ignoran. También se pueden separar celdas con coma, por ejemplo =SUMA (A1, B2) .

Los **números negativos** se muestran en color rojo.

7. Moverse con el teclado

Tecla	Qué hace
Flechas	Mueven la celda seleccionada
Enter	Con una celda seleccionada, empieza a editarla. Al escribir, guarda y baja una celda
Tab	Guarda y pasa a la celda de la derecha
Esc	Cancela lo que se estaba escribiendo
Supr o Retroceso	Borra el contenido de la celda seleccionada
Cualquier letra o número	Empieza a escribir en la celda seleccionada

8. Mensajes de error

Si algo sale mal, la celda no se rompe: muestra un mensaje en rojo.

Mensaje	Qué significa	Qué hacer
#DIV/0!	Se intentó dividir entre cero, o se pidió un promedio sin números	Revise el número por el que divide o las celdas del rango
#ERROR!	La fórmula está mal escrita (falta un paréntesis, hay un símbolo que no se permite...)	Edite la celda y corrija la fórmula
#REF!	La fórmula usa una celda que no existe en la hoja (por ejemplo Z99)	Use columnas de la A a la J y filas de la 1 a la 15
#VACIO!	La fórmula usa una celda que está vacía	Escriba un valor en esa celda
#VALUE!	La fórmula intenta hacer una cuenta con texto	Revise que las celdas usadas tengan números
#NAME?	Se escribió una función que no existe	Use SUMA, PROMEDIO, MAX o MIN
#CIRCULAR!	Las celdas se usan entre sí en círculo (A1 usa B1 y B1 usa A1)	Cambie una de las fórmulas. Arriba de la tabla se indica qué celdas son

9. Guardar y exportar

- **Guardado automático:** todo lo que escribe se guarda solo en el navegador de su computadora. Si cierra la página y la vuelve a abrir, todo sigue ahí. Si borra los datos de navegación o usa otra computadora, no lo verá.
- **Exportar CSV:** presione el botón. Se descarga el archivo **hoja.csv** con los resultados (no las fórmulas), que puede abrirse en Excel.
- **Limpiar hoja:** borra todo el contenido. Antes de hacerlo pide confirmación.

5.6 Bitácora

Nivel 1)

Para el nivel 1 nos pide generar de una forma dinámica, con JavaScript, tablas de HTML, con sus respectivos encabezados cada de columna tipo Excel, (A, B, C...), el objetivo es demostrar el dominio de la generación dinámica de elementos DOM a partir de estructuras de control (ciclos anidados).

En nuestro documento de HTML vamos a usar `<table>` y `<script>` para las tablas.

En el `<script>` vamos a definir dos constantes con el numero de columnas y filas, esto nos ayudara para cuando deseemos cambiar el tamaño de la hoja.

Para las columnas necesitamos que los números se vuelvan letras, escribiremos una función para eso

Usaremos `"String.fromCharCode(65 + x)"` que devuelve una letra por un numero

por ejemplo 0 = A, 1= B...

Ahora para los encabezados de las filas, usaremos `"for"` para que recorra las columnas. Las letras son los encabezados de columna; los números de fila vienen después. Por cada una, creara un `<th>` "con su letra y lo agregara a una fila `<tr>`"

Usaremos “document.createElement('th')”, “.textContent = ... y .appendChild(...)”

Ahora usaremos los ciclos anidados:

¿Qué son los ciclos anidados?
Básicamente es un ciclo que esta dentro de otro ciclo, y por cada vuelta del ciclo de afuera, el ciclo de adentro corre completo, de principio a fin.

Para las filas de datos, usando un “for” que recorra las filas y, dentro de cada vuelta, otro “for” que recorra las columnas.

Por cada fila: creara un “<tr>”, le agregara un <th> con el numero de fila (empezando en 1, no en 0) y luego una “<td>” por cada columna.
Cada fila terminada se agrega al “<tbody>”

Nivel 2) Estado y edición de celdas

Nos pide que se guarden los datos y que se muestren en las tablas.

Crearemos un objeto vacío y lo llamaremos datos. Usaremos como clave el nombre de la celda y como valor el texto que escribió el usuario.

Decisión	Razón
Guardar los datos en un objeto datos, con el nombre de la celda como clave (datos["A1"] = "hola")	Es simple de leer y de guardar, y no ocupa espacio por celdas vacías.
Guardar solo el texto que escribió el usuario	Más adelante los resultados de las fórmulas se calcularán aparte, sin perder lo que escribió.
Mantener un arreglo celdas[y][x] con los elementos <td>	Permite encontrar cualquier celda sin buscarla en el DOM.
Un solo evento dblclick en el <tbody>	Evita crear 150 eventos, uno por celda.
Mostrar el texto con textContent y no con innerHTML	Evita que el texto del usuario se interprete como código HTML.

Problemas encontrados

El campo de edición no quedaba sobre la celda. Se soluciono con “ position: relative” y en las celdas “ position: absolute; inset: 0” , en el input.

Nivel 3) Las fórmulas:

Acá usamos la primera parte del tokenizador, ya que separamos los datos para luego en el evaluador hacer las operaciones.

En la celda, cuando se empieza con “ = ”, se introducen los datos para luego ya arrojar los resultados, por ejemplo cuando introducimos “ =3+3*4 “ nos va a mostrar “ 15 “.

El tokenizador recorre el texto letra por letra y lo corta en pedacitos. Si ve un dígito, lee el número completo; si ve una letra, lee el nombre completo (como A1 o SUMA); si ve un símbolo (+, -, *, /, paréntesis), lo guarda. Si ve cualquier otro carácter, muestra #ERROR!. Así, 3+3*4 queda como: 3, +, 3, *, 4.

El evaluador lee esos pedacitos con cuatro funciones, una por cada nivel de prioridad: expr (sumas y restas), term (multiplicaciones y divisiones), unario (el signo negativo) y primario (números y paréntesis). Como expr le pasa el trabajo a term, las multiplicaciones siempre se resuelven antes que las sumas. Por eso 3+3*4 da 15 y no 24.

Como no usamos la función “ eval() ”, primero se separa cada uno de los datos, y luego se resuelven en el orden correcto, en este caso sería primero la multiplicación y luego las sumas y restas.

Problema: al probar el código con el que empecé, vi que =1+2) daba 3 y no error, y que =-5 daba error. Lo corregí agregando una revisión al final y una función para el signo negativo.

El evaluador lee de izquierda a derecha:

1. Lee el 1.
2. Ve el + y lee el 2.
3. Calcula $1 + 2 = 3$.
4. Se detiene, porque no encuentra ningún operador más.

El “)” nunca lo leyó y quedó sin usar. Como nadie lo revisó, el programa devolvía 3 como si todo estuviera bien.

La solución:

Al final del evaluador, comprobar si quedaron pedacitos sin leer:

```
if (pos < tokens.length) throw new Error('#ERROR!');
```

Aquí pos es la posición del pedacito que toca leer, y tokens.length es cuántos pedacitos hay en total.

En el ejemplo, el evaluador terminó con pos = 3 y hay 4 pedacitos "tokens.length = 4".

Como $3 < 4$, sobró algo, así que muestra #ERROR!.

Lo que aprendí: un problema difícil se vuelve fácil si lo divido en pasos pequeños.

Nivel 4) Usar otras celdas

Como ahora puedo escribir fórmulas como =A1+B2. Si cambio el valor de A1, el resultado se actualiza solo.

Cómo lo hice: para calcular una celda, primero pregunto por las celdas que necesita. Cada vez que algo cambia, el programa recalcula toda la hoja.

Nivel 5) Funciones

Agregué SUMA, PROMEDIO, MAX y MIN. Por ejemplo, =SUMA(A1:A3) suma desde A1 hasta A3.

Cómo lo hice: convierto el rango A1:A3 en una lista de celdas: A1, A2 y A3. Luego me quedo solo con los números y hago la cuenta. El texto y las celdas vacías no cuentan.

Nivel 6) Los errores

Si algo sale mal, la celda muestra un mensaje en rojo:

- #DIV/0!: dividí entre cero.
- #ERROR!: la fórmula está mal escrita.
- #REF!: usé una celda que no existe.
- #VACIO!: usé una celda vacía.
- #VALUE!: intenté sumar una palabra.
- #NAME?: usé una función que no existe.
- #CIRCULAR!: hay un círculo (explicado abajo).

Círculo sin fin: si A1 dice =B1 y B1 dice =A1, nunca terminarían de calcularse. Para evitarlo, llevo una lista de las celdas que estoy calculando en ese momento. Si me piden una celda que ya está en la lista, sé que hay un círculo, muestro #CIRCULAR! y paro. Así la página no se congela.

Nivel 7) Guardar y exportar

Todo lo que escribo se guarda en el navegador (localStorage). Si cierro la página y la abro otra vez, todo sigue ahí.

El botón Exportar CSV descarga un archivo con los resultados, que se puede abrir en Excel.

El botón Limpiar hoja borra todo, y antes me pregunta si estoy seguro.

Nivel 8) Dos mejoras

1. **Moverme con el teclado:** las flechas mueven la celda seleccionada, Enter edita, Tab pasa a la derecha, Esc cancela y Supr borra.
2. **Números negativos en rojo:** si el resultado es menor que cero, se muestra en rojo.

Conclusión

Para poder solucionar este tipo de prácticas se requiere de mucho análisis y mucha práctica, al final hay muchas soluciones para los mismos problemas, pero cada uno tiene su complejidad.

Sobre escribir código, considero que aún me falta mucho por aprender, pienso que uno como estudiante siempre debe de estar dispuesto a aprender cosas nuevas y de investigar por su propia cuenta, es difícil, pero eso es lo que nos hace aprender, no puedo asegurar que me sé el código a memoria, pero creo que logre darle la mejor solución y poder entender lo que estaba haciendo en esta práctica y al momento de escribir el código.

VIDEO MOSTRANDO EL FUNCIONAMIENTO DEL CODIGO.

https://drive.google.com/file/d/1SdOt3El9l25RWOOn9lUEOxtH_4mdem5mX/view?usp=sharing

